```
{
  "name": "react-interview",
  "private": true,
  "version": "1.0.0",
  "type": "module", #A
}
```

This setting makes Node.js treat `.js` files as ES modules, allowing import and export syntax. Without it or if set to `"commonjs"`, `.js` files are treated as CommonJS. Files with `.mjs` are always ES modules, and `.cjs` files are always CommonJS.

ES modules support circular dependencies but with important limitations. When modules import each other, the JavaScript engine initializes each module up to its export statements before resolving imports. If a module accesses an imported value from a dependency that hasn’t finished initializing, that value may be incomplete or undefined, potentially causing subtle bugs during module initialization. Therefore, it is best practice to avoid circular dependencies whenever possible.

For this topic, are you able to answer these follow-up/probing questions?

- "What are the benefits of using ES modules over older patterns like IIFE or CommonJS, especially in large-scale applications?"
- "How do dynamic imports work in ES modules?"
- "What is top-level `await` in ES modules, and how does it change module behavior compared to older JavaScript versions?"
- "How do ES modules handle circular dependencies?"
- "In what ways do ES modules improve security and maintainability compared to global script patterns?"

# 3.5 Jane's JavaScript interview skill assessment

Frontend interviews often include challenges that combine JavaScript fundamentals with practical problem-solving. These exercises test more than just language concepts—they assess a candidate’s ability to apply their knowledge to real-world projects and build functional, maintainable solutions.